

ANALYSIS REPORT
BLUE PROJECT

CoDrive

Author(s):

Famura

Contents

1. Project Description.....	3
2. Planed Project Steps	3
2.1. Reverse Engineering.....	3
2.2. UI	3
2.3. Dependency Structures.....	4
2.4. Risk Engine	5
2.5. Prolouge	5
2.6. AI	5
3. Conclusion	6
4. Credits.....	6
5. Referances.....	7
6. Disk Content	7

1. Project Description

The CoDrive project has been planned as a complete redevelopment by enhancing and improving the E-Sys application. The E-Sys application was originally developed by engineers working at BMW, not for everyday users, but for other engineers and technicians within the BMW factory. However, after a built version of the application was leaked, it became publicly available. As a result, the application is quite outdated in terms of readability and usability. The CoDrive project is therefore designed to address these issues and introduce new features.

The biggest challenge in the project is that, although E-Sys is publicly available, its source code is not open. Understanding how the application works is crucial in order to make meaningful progress.

2. Planed Project Steps

2.1. Reverse Engineering

As stated in the description, the source code of the project is not publicly available. There are significant differences between an application that merely retrieves data from the ECU and one that can access and modify the ECU's embedded code. All stages—such as determining which protocols the application uses to communicate with the ECU and identifying the additional signals sent for code modification—must undergo a reverse engineering process. This step should be carried out as the initial phase; however, once certain milestones are achieved, the process can continue in an Agile manner.

- This step planned as approximately 1 month.
- For the start, these links may be useful:
 - [Github](#), [Reddit](#), [Reddit](#)

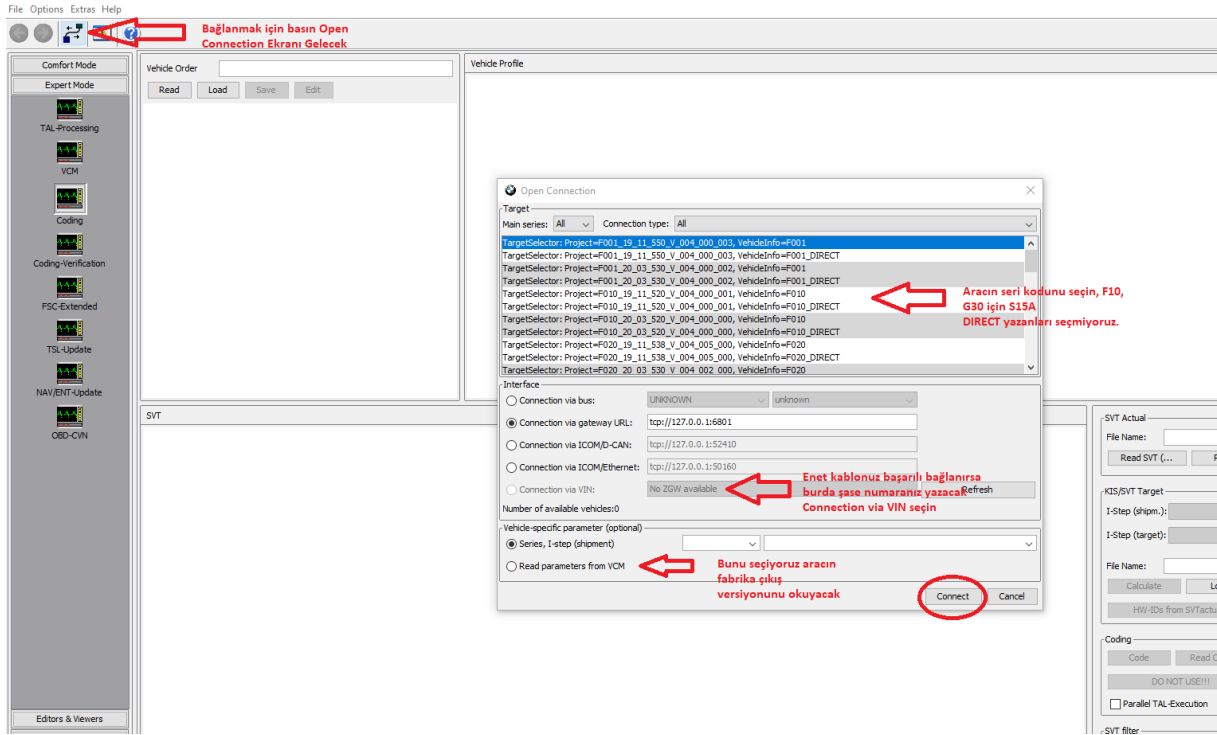
Since this phase does not have a well-defined or standardized structure, working with someone experienced in reverse engineering would make the process significantly easier. It is likely that a person without prior experience in this field may struggle and either fail to achieve success or require a considerable amount of time to do so.

2.2. UI

Daha önce belirtildiği üzere uygulamanın güncel UI yapısı modernizasyona ihtiyaç duymalıdır. Uygulamanın nasıl çalıştığı, ekrandaki bölümlerin önemi ve eklenecek yeni özelliklerle beraber genel erişilebilirlik öncelikleri belirlenip buna göre bir hiyerarşi yapısı hazırlanmalıdır. Bu hiyerarşi yapısına göre de kullanıcı dostu bir UI tasarlanmalıdır. Bu UI'nin en azından prototipinin bundan sonraki aşamalardan önce oluşması önemlidir. Bu şekilde AI gibi maddelerde bildirim gibi durumların nasıl halledileceği önceden belirlenmiş olmalıdır.

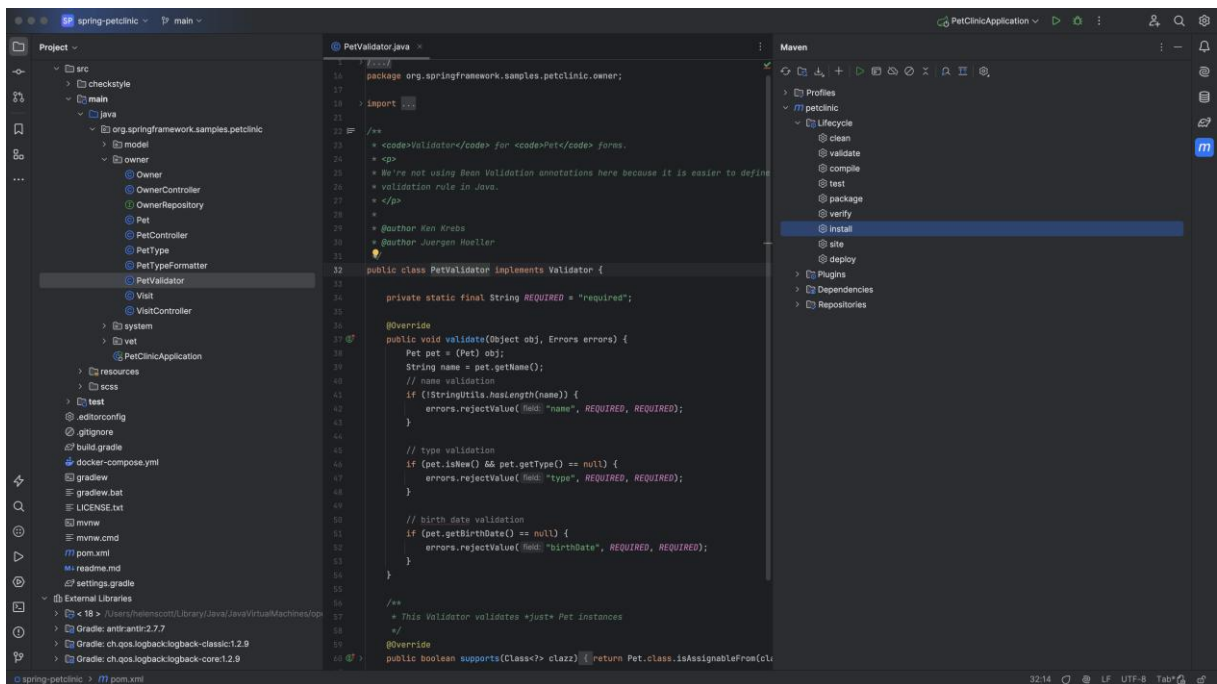
- This step planned as approximately 2 weeks:
 - 2 days for Hierachic structure,
 - 8 days for the prototype of the UI

A screenshot for E-Sys can be found below:



2.3. Dependency Structures

In the E-Sys application, compatibility issues arising from the versions of libraries and tools used within the ECU are very common and often require significant time and patience to resolve. These dependencies should be taken out of the user's control. Leaving them under user control can lead to users abandoning the application before they are able to use it, and may also cause those who do use it to approach the application with strong bias. Therefore, a dependency management and download system similar to the collaboration between Java, Maven, and IntelliJ IDEA should be established.



Dependencies such as the Java version in use and the required libraries should be downloadable with a single action—similar to how Maven handles Java—and made ready for development automatically.

- This step planned as approximately 3 weeks.
- For the start, these links may be useful:
 - [Github](#), [Intelij IDEA](#)

2.4. Risk Engine

The application should include a general risk engine capable of identifying potentially critical or fatal changes to the vehicle before they are applied, and warning the user accordingly. To achieve this, error levels must first be defined. Based on these levels, issues should be categorized in advance as minor errors, significant errors, and errors that could prevent the vehicle from operating. These should then be communicated clearly to the user. In this way, the user can receive feedback under certain conditions before causing any damage to the vehicle.

- This step planned as approximately 3 weeks.
- During the testing phase of this stage, any modifications made on the ECUs should be carried out under the supervision of—or directly by—someone who has prior experience with E-Sys. There is a high risk that the developer may inadvertently cause damage to the vehicle during testing.
- At this stage, it is crucial to research commonly reported issues shared on forums as a guiding resource. For example:
 - [TechRoute66](#), [Reddit](#), [Bimmerpost](#)

2.5. Prolouge

There is a considerable amount of instructional content about E-Sys available on YouTube. However, leaving users in a “learn or drown” situation and forcing them to rely on external resources for learning is not suitable by modern standards. Therefore, when the application is first installed, it should include additional inline comments within the ECU code and guide the user through a few simple actions step by step within the application itself. In this way, the goal is to provide a better first impression—where users most commonly abandon applications—and to increase both user retention and continued usage rates.

- This step planned as approximately 2 weeks.

2.6. AI

It is planned to integrate an interactive AI system into the application. This AI system should actively monitor the vehicle’s parameters both while the car is running and when it is not, detect potential faults, and provide possible solutions. It should also assist the user in performing specific modifications and include a voice interaction feature. Since it will operate on the ECU, it must be highly optimized. Furthermore, after the product is released, the AI should continue to improve using live user data. The necessary legal considerations regarding data processing and

the communication methods to be used have been excluded from the initial development scope and therefore need to be researched.

- This step planned as approximately 2 weeks:
 - 1 month for training AI model,
 - 1 month for optimization.
- The use of Neural Networks is recommended for this phase. Considering the complex structure of a vehicle, it appears to be the most effective algorithm.
- Optimization is extremely important at this stage. If optimization is not handled properly, issues such as ECU freezing or excessive power consumption may occur. This could indirectly cause damage to the ECU or other components. Additionally, identifying that the issue originates from the AI may take a long time, during which permanent damage could occur.

3. Conclusion

The CoDrive project is a comprehensive redevelopment initiative aimed at overcoming the limitations of the existing E-Sys application and bringing the user experience up to modern standards. One of the most critical aspects highlighted throughout the project is that, due to the closed-source nature of the application, reverse engineering becomes a fundamental requirement. While this increases the technical difficulty of the project, it also presents an important opportunity to gain a deeper understanding of the system.

An analysis of the planned phases shows that the project is not merely a technical rewrite; it also includes significant improvements in terms of user experience, accessibility, and security. In particular, components such as UI modernization, automated dependency management, and the risk engine will help make the application both more stable and more user-friendly.

In addition, the planned integration of an AI system has the potential to transform CoDrive from a simple vehicle coding tool into an intelligent support system. However, it is important to note that performance optimization and security are critical factors at this stage.

In conclusion, if executed with proper planning and expertise, the CoDrive project has the potential to fill an important gap within the current E-Sys ecosystem and serve as a strong alternative for both technical users and beginners. The success of the project will largely depend on the efficient management of the reverse engineering process and the safe testing of the developed systems.

4. Credits

- Famura
- Mustafa Kilic

5. References

- Admin, T. (2026, March 11). Master BMW coding with E-SYS: Complete beginner's guide. TechRoute66. https://techroute66.com/bmw-e-sys-the-ultimate-guide-for-beginners/?srsltid=AfmBOooFIQZrgh6IGQTBkOye3NAopA4NOULXXVOj9cwZKw5hQdydh8Z_
- ESYS ENET KODLAMA F & G serileri | DonanimHaber Forum. (2020, April 8). DonanimHaber Forum. <https://forum.donanimhaber.com/esys-enet-kodlama-f-g-serileri--142775407>
- JetBrains. (2023, February 17). The Maven Tool Window. The Maven Tool Window RSS. <https://www.jetbrains.com/guide/java/tutorials/working-with-maven/maven-tool-window/>
- BimDoc. (n.d.). BMW coding is easier than you think. Full E-Sys coding guide. Part 1. YouTube. <https://www.youtube.com/watch?v=FaZXx49UxJY>
- GitHub. (n.d.). GitHub: Build and ship software. <https://github.com>
- Reddit. (n.d.). Reddit: Dive into anything. <https://www.reddit.com>
- Bimmerpost. (n.d.). Bimmerpost BMW forums. <https://www.bimmerpost.com>
- TechRoute66 Forum. (n.d.). BMW coding discussions and troubleshooting. <https://techroute66.com>
- Oracle. (n.d.). Java documentation. <https://docs.oracle.com/en/java/>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. <https://www.deeplearningbook.org>

6. Disk Content

- “BMW coding is easier than you think. Full E-Sys coding guide. Part 1” Youtube videosu.
- This analysis on .docx format.
- This analysis on .pdf format.
- Hart Attack(Metal Ver.) by Syed